

Testing

Testing Strategy & Specification

This document describes the testing architecture, test suites, coverage metrics, and mock infrastructure for UnifyVault V2 across both smart contracts (`packages/protocol`) and the web application (`apps/web-v2`).

1. Overview

UnifyVault V2 implements a defense-in-depth testing strategy:

- **Foundry** (`forge`): Used for smart contract unit testing, fuzzing, invariant testing, and adversarial attack simulation.
- **Vitest**: Used for frontend unit testing, math transformations, oracle multi-state refresh testing, marketplace decimals validation, and cryptographic payment verification testing.

2. Smart Contract Test Suites (`packages/protocol`)

Test files are located in `packages/protocol/test`:

```
packages/protocol/test/
├── unit/
│   ├── ProtocolDirectory.t.sol           # Registry unit tests
│   ├── UnifyVaultController.t.sol        # Controller unit & integration tests
│   ├── CustodyVault.t.sol                # Vault balance & donation immunity tests
│   ├── Treasury.t.sol                   # Fee collection & treasury tests
│   ├── FeeManager.t.sol                  # Fee calculation tests
│   ├── LiquidityManager.t.sol            # Liquidity threshold & sweep tests
│   ├── OracleManager.t.sol               # Price aggregator & staleness tests
│   ├── ChainlinkOracleProvider.t.sol     # Chainlink feed adapter tests
│   ├── StrategyManager.t.sol             # Target allocation BPS tests
│   ├── PortfolioManager.t.sol            # NAV calculation & rebalance tests
│   ├── SwapAdapter.t.sol                 # DEX router & slippage tests
│   ├── UVBEV2.t.sol                     # Share token mint/burn & hook tests
│   ├── CostBasisManagerV2.t.sol          # Cost basis conservation & P2P isolation tests
│   ├── P2PEscrow.t.sol                   # P2P Escrow lifecycle & settlement tests
│   ├── P2PEscrowAdversarial.t.sol        # Reentrancy & unauthorized release tests
│   ├── Marketplace.t.sol                 # Limit orderbook matching tests
│   └── UnifyVaultTimelock.t.sol           # 48h timelock governance tests
├── invariant/
│   ├── Handlers.sol                     # System invariant test suite
│   ├── ProtocolInvariants.t.sol
│   └── P2PEscrowInvariant.t.sol          # Solvency and balance match invariants
├── fuzz/
│   └── RedemptionFuzz.t.sol              # Property-based fuzzing tests
├── fork/
│   └── MainnetDeploymentValidation.t.sol  # On-chain fork validation
```

3. Frontend Test Suites (`apps/web-v2`)

Located across `apps/web-v2`:

- **oracleFeedRefresh.test.ts**: Verifies multi-query atomic refresh, staleness error handling, and deviation guard preservation.

- `**`portfolioTransforms.test.ts`**`: Verifies graceful ``Price unavailable`` and ``Value unavailable`` rendering for non-live feeds.
- `**`marketplaceDecimals.test.ts`**`: Verifies exact decimal scaling and price calculations for USDC (6), cbBTC (8), WETH (18), and UVBE (18).
- `**`paymentVerification.test.ts`**`: Verifies single-authority payment verification, replay prevention, and wallet signature authentication.

—

4. Test Execution Commands

4.1 Run Smart Contract Tests

```
# Run all Foundry tests
cd packages/protocol && forge test

# Run P2P Escrow & Marketplace test suites
forge test --match-contract "P2P"

# Run CostBasisManager tests
forge test --match-path "test/unit/CostBasisManagerV2.t.sol"

# Run with gas report
forge test --gas-report
```

4.2 Run Frontend Tests

```
# Run all Vitest suites
cd apps/web-v2 && npx vitest run

# Run specific test suite
npx vitest run hooks/__tests__/oracleFeedRefresh.test.ts
```